Github Repository Link
https://github.com/ahmede06/legends-sword-wand

1. System Overview

This project will implement the game: Legends of Sword and Wand as a cloud system using a microservices architecture.

Instead of just putting the entire game into one large application, we will separate the game into independent services, and each service will then handle one major aspect of the game.

The system includes the following services:

- User Service

- Party Management Service

- Battle Service

- Campaign Service

- API Gateway

Each service runs inside its own Docker container, which will allow them to run on their own and also scale if needed.

All of the services communicate through HTTP REST APIs, and the API Gateway is the main entry point of the client interface.

The player's data, party information, progress, and scores are stored in a database so that the player can log back in and continue from where they left off.

2. System Architecture

Our system will use microservices as our architecture, where every part of the game will be implemented as separate services from one another.

Our architecture includes all of these components:

- Client GUI
- API Gateway
- User Service
- Party Service
- Battle Service
- Campaign Service
- Database

The client GUI is going to send requests to the API Gateway, which will then route these requests to the correct service.

All of these services will perform their own specific task and return their responses.

For example:

User login → API Gateway → User Service
Start campaign → API Gateway → Campaign Service
Battle action → API Gateway → Battle Service

This architecture will make our system more modular and easier to maintain and control.

3. Services and Responsibilities

User Service

The User Service will manage the player accounts and the authentication

The User Service will also let players register and log into the game. The service will store all of the usernames and passwords of the users inside the database

The main purpose of this service is to ensure that every player will have a persistent profile where things like progress, saved parties, and scores are stored.

Responsibilities:

- Register new users
- Authenticate logins
- Store player profile information
- Connect player data to their account

Endpoints:

POST /register
This will create a new user account.

POST /login
This will authenticate the user and then allow them to access the system.

Party Management Service

The Party Service manages all the heroes and their party-related functions

Players can do things like recruit their own heroes, organize their party, and manage their character's attack, defense, health, and mana stats.

A party can contain between 1 and 5 heroes.

Party Management Service

Heroes belong to classes such as Order, Chaos, Warrior, or Mage, and they gain experience after battles.

Responsibilities:

- Create and manage parties

- Add or remove heroes

- Store hero stats and levels

- Provide party info to other services

Endpoints:

POST /create-party – This will create a new party

GET /party/{username} – This returns party info

POST /add-hero – This adds a hero

Battle Service:

Handles combat between parties (PvE or PvP). Calculates damage and updates stats.

This service handles the PvP and PvE combat that happens in the game. It calculates damage and then updates the stats of the damage.

Damage formula:

Damage = Attack − Defense

Responsibilities:

- Start battles

- Process actions

- Update HP and mana

- Determine results

Endpoints:

POST /start-battle

POST /attack

POST /defend

POST /cast

Campaign Service (PvE)

The player will explore a dungeon. Rooms can be battles or inns. After 30 rooms, the final score will be calculated.

Responsibilities:

- Generate rooms

- Track the player's progress

- Calculate score

- Save state

Endpoints:

POST /start-campaign

GET /next-room

POST /end-campaign

API Gateway

Single entry point that will route the requests to services.

Responsibilities:

- Route requests

- Simplify communication

- Request flow:

Client → API Gateway → Service

4. Service Communication

All of the services will communicate using REST APIs

When a user performs an action in the client interface, the request is sent to the API Gateway.

Whenever the player will perform an action in the client interface, a request will be sent to the API Gateway. The API Gateway will then determine which service should handle the request and forward it accordingly.

For example:

User logs in
Client → API Gateway → User Service

User starts PvE campaign
Client → API Gateway → Campaign Service

Battle begins
Campaign Service → Battle Service

This communication model will ensure that all of the services remain independent and loosely coupled.

5. Data Storage

The system will use a database that stores all the game data

This includes:

User accounts

Player parties

Hero statistics

Campaign progress

Scores and rankings

Storing this data will allow for players to leave the game and then continue playing it later without them losing any of their progress from before.

All these services will access the database through their APIs, ensuring that no data will be stored on the client side.

6. Testing

Unit tests are implemented so that we can verify that the core functionality for every service works correctly.

Examples of unit tests include:

- User Service

- Test user registration

- Test login authentication

- Test invalid login credentials

- Party Service

- Test creating a party

- Test adding heroes

- Test retrieving party data

- Battle Service

- Test damage calculation

- Test attack actions

- Test defending actions

- Campaign Service

Test campaign creation

Test room generation

Test score calculation

All these tests help ensure that every service will behave correctly and prevent errors when new changes are introduced.

7. Docker Deployment

Each service is packaged into a docker container.

Docker will allow for the services to run in isolated environments that have all the required dependencies.

We use Docker Compose to start all services together

The docker-compose file defines the containers for:

- User Service
- Party Service
- Battle Service
- Campaign Service
API Gateway

Running the system will require just one command which is:

docker-compose up

This command will automatically build and start all the services

8. CI/CD Pipeline

A CI/CD pipeline was implemented so that the testing and deployment can be automated.

The pipeline will perform these steps:

- Pull the latest code from the GitHub repository.

- Run automated unit tests for every service.

- Build Docker images for the services.

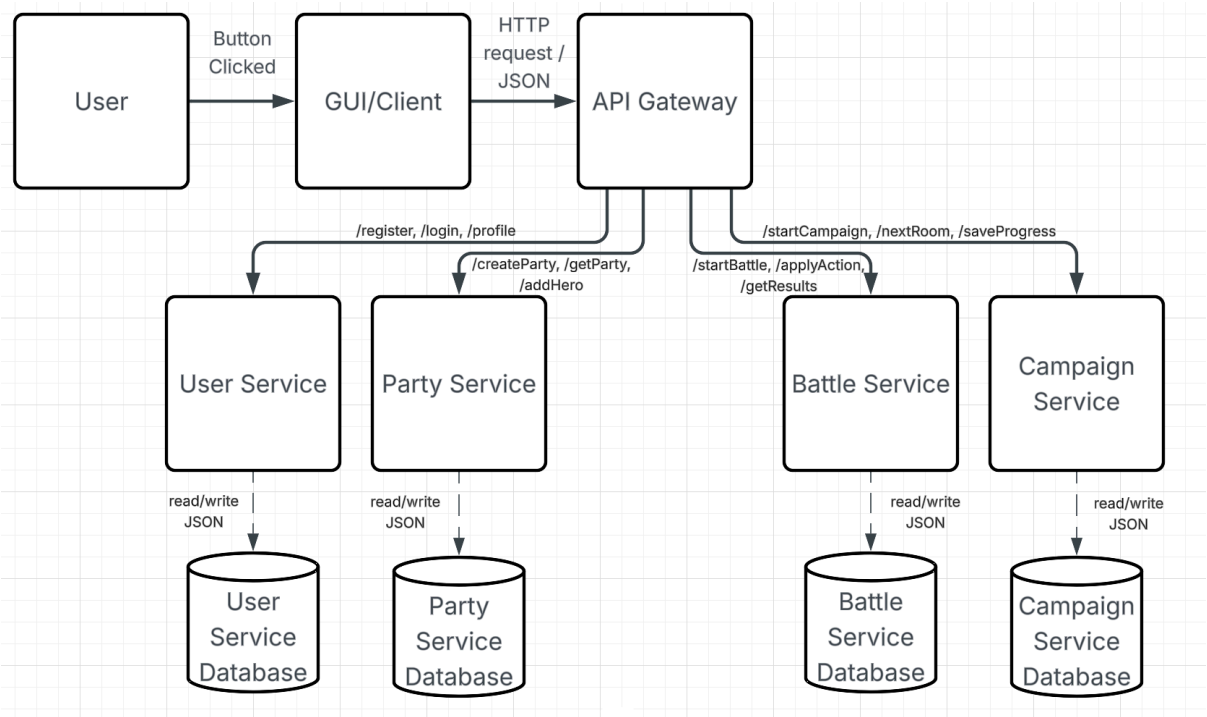
- Deploy the containers.

This will ensure that new changes will be automatically tested and then deployed without needing any manual help.

9. Conclusion

Our project shows how a complex game can be implemented using a microservices architecture. By using independent services, it becomes easier to code while also maintaining and scaling our system. Using Docker containers and the CI/CD automation ensured that the system was able to be deployed consistently. Using all these things helped us create a strong base for a working RPG game.

Diagram


User
Button Clicked
GUI/Client
HTTP request / JSON
API Gateway
/register, /login, /profile
/createParty, /getParty, /addHero
/startBattle, /applyAction, /getResults
/startCampaign, /nextRoom, /saveProgress
User Service
Party Service
Battle Service
Campaign Service
read/write JSON
read/write JSON
read/write JSON
read/write JSON
User Service Database
Party Service Database
Battle Service Database
Campaign Service Database